

Creación de Layouts mediante Programación en Android Studio

Existen dos formas principales de construir interfaces gráficas (UI):

1. **Mediante XML** (la más común).
2. **Mediante programación en Java o Kotlin** (creando las vistas dinámicamente).

Cuando hablamos de **crear layouts mediante programación**, nos referimos a generar los componentes visuales desde el código en tiempo de ejecución, sin definirlos previamente en un archivo XML.

1. ¿Qué es un Layout? Un **Layout** es un contenedor que organiza los elementos visuales de una pantalla.

Por ejemplo:

- Botones
- Textos
- Imágenes
- Campos de entrada

Todos estos componentes necesitan un contenedor que indique:

- dónde se colocan,
- cuánto espacio ocupa,
- cómo se alinean.

Ejemplos de layouts:

- [LinearLayout](#)
- [RelativeLayout](#)
- [ConstraintLayout](#)
- [FrameLayout](#)

2. Creación tradicional mediante XML

Normalmente hacemos algo así:

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:text="Hola Mundo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
</LinearLayout>
```

Android carga este XML y construye automáticamente los objetos visuales.

3. Creación mediante Programación

En lugar de XML, podemos crear los objetos nosotros mismos.

Ejemplo:

```
LinearLayout layout = new LinearLayout(this);
```

```
TextView texto = new TextView(this);  
texto.setText("Hola Mundo");
```

```
layout.addView(texto);
```

```
setContentView(layout);
```

Aquí estamos creando toda la interfaz desde Java.

4. Análisis detallado línea por línea

Línea 1

```
LinearLayout layout = new LinearLayout(this);
```

¿Qué hace?

Crea un objeto LinearLayout.

Desglose

LinearLayout

Es la clase que representa el contenedor.

LinearLayout

Significa:

"Quiero un layout que organice elementos en línea."

Puede ser:

- vertical
- horizontal

Layout: layout

Es el nombre de la variable.

Podría llamarse:

contenedor
principal
miLayout

new

new

Reserva memoria y crea el objeto.

Es decir:

"Construye un nuevo LinearLayout."

this

this

Hace referencia al contexto actual.

En una Activity:

this

significa:

"La pantalla actual."

Android necesita el contexto para:

- acceder a recursos,
- temas,
- estilos,
- configuración.

Línea 2

```
TextView texto = new TextView(this);
```

¿Qué hace?

Crea una etiqueta de texto.

Desglose

TextView

Componente visual para mostrar texto.

TextView

Ejemplos:

- títulos
- mensajes
- etiquetas

texto

Nombre de la variable.

texto

Representa el TextView creado.

new TextView(this)

Crea físicamente el componente.

Línea 3

```
texto.setText("Hola Mundo");
```

¿Qué hace?

Asigna contenido al TextView.

Desglose

setText()

Método que cambia el texto mostrado.

setText()

Significa:

"Muestra este texto en pantalla."

"Hola Mundo"

Cadena de caracteres.

"Hola Mundo"

Será el contenido visible.

Línea 4

```
layout.addView(texto);
```

¿Qué hace?

Añade el TextView al LinearLayout.

Desglose

addView()

Método que inserta una vista dentro de otra.

addView()

Significa:

"Coloca este componente dentro del contenedor."

Antes:

Layout vacío

Después:

Layout

└─ TextView

Línea 5

```
setContentView(layout);
```

¿Qué hace?

Convierte ese layout en la pantalla visible.

Desglose

setContentView()

Método de Activity.

Indica:

"Esta será la interfaz de usuario."

5. Configurando Propiedades del Layout

Por defecto no sabemos cómo se organizarán los elementos.

Debemos configurar el layout.

```
layout.setOrientation(LinearLayout.VERTICAL);
```

Desglose

setOrientation()

Define la orientación.

Opciones:

LinearLayout.VERTICAL

o

LinearLayout.HORIZONTAL

Vertical

VERTICAL

Resultado:

Texto1

Texto2

Texto3

Horizontal

HORIZONTAL

Resultado:

Texto1 Texto2 Texto3

6. Definiendo Tamaños

Todo componente necesita tamaño.

Android utiliza:

MATCH_PARENT

WRAP_CONTENT

MATCH_PARENT

MATCH_PARENT

Significa:

Ocupa todo el espacio disponible.

WRAP_CONTENT

WRAP_CONTENT

Significa:

Ocupa sólo el espacio necesario.

Ejemplo

```
LinearLayout.LayoutParams parametros =  
    new LinearLayout.LayoutParams(  
        LinearLayout.LayoutParams.MATCH_PARENT,  
        LinearLayout.LayoutParams.WRAP_CONTENT  
    );
```

Explicación completa

LayoutParams

Objeto que almacena reglas de tamaño.

Primer parámetro

MATCH_PARENT

Ancho completo.

Segundo parámetro

WRAP_CONTENT

Altura mínima necesaria.

Aplicando los parámetros

```
texto.setLayoutParams(parametros);
```

Significa:

"Utiliza estas reglas de tamaño."

7. Agregar un Botón

```
Button boton = new Button(this);
```

Crea un botón.

```
boton.setText("Aceptar");
```

Texto del botón.

```
layout.addView(boton);
```

Lo inserta en el layout.

Resultado:

```
+-----+
| Hola Mundo      |
|                |
| [Aceptar]       |
+-----+
```

8. Eventos de Botón

Un botón normalmente responde a clics.

```
boton.setOnClickListener(
    new View.OnClickListener() {
        @Override
        public void onClick(View v) {

        }
    }
);
```

Explicación

setOnClickListener()

Registra un escuchador de clics.

Significa:

"Ejecuta código cuando el usuario pulse."

onClick()

Método que se ejecuta al pulsar.

```
public void onClick(View v)
```

- public → accesible
- void → no devuelve nada
- View v → componente pulsado

Ejemplo completo

```

boton.setOnClickListener(
    new View.OnClickListener() {
        @Override
        public void onClick(View v) {

            texto.setText("Botón pulsado");

        }
    }
);

```

Flujo:

```

Usuario pulsa
  ↓
onClick()
  ↓
setText()
  ↓
Texto actualizado

```

9. Ejemplo Completo para Mostrar en Clase

```

package com.example.layoutprogramado;

import android.os.Bundle;
import android.widget.Button;
import android.widget.LinearLayout;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        LinearLayout layout = new LinearLayout(this);

        layout.setOrientation(
            LinearLayout.VERTICAL);
    }
}

```



```

        TextView texto = new TextView(this);

        texto.setText(
            "Ejemplo de Layout Programado");

        Button boton = new Button(this);

        boton.setText("Cambiar Texto");

        boton.setOnClickListener(v -> {

            texto.setText(
                "Texto modificado");

        });

        layout.addView(texto);

        layout.addView(boton);

        setContentView(layout);
    }
}

```

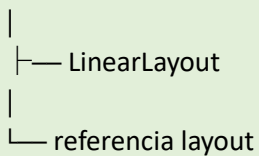
Diagrama de Memoria

Cuando se ejecuta:

```
LinearLayout layout = new LinearLayout(this);
```

Android crea:

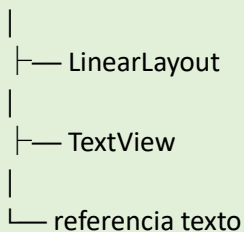
Memoria



Luego:

```
TextView texto = new TextView(this);
```

Memoria



Después:

```
layout.addView(texto);
```

Relación:

LinearLayout



TextView

Finalmente:

```
setContentView(layout);
```

Pantalla



LinearLayout



— TextView



— Button

Ventajas de crear Layouts por Programación

Interfaces dinámicas. Crear controles en tiempo real. Formularios generados automáticamente. Pantallas que cambian según datos recibidos. Mayor flexibilidad.

Desventajas

Código más largo. Menos visual. Más difícil de mantener. ConstraintLayout es más complejo por código.

Conclusión: La creación de layouts mediante programación consiste en **instanciar objetos visuales (Views), configurar sus propiedades, añadirlos a contenedores (Layouts) y finalmente mostrarlos mediante setContentView()**. Cada componente que vemos en pantalla es un objeto Java/Kotlin que Android dibuja siguiendo una jerarquía de vistas (View Tree). Comprender este proceso ayuda a entender cómo Android construye internamente una interfaz, incluso cuando trabajamos con XML o con tecnologías más modernas como **Jetpack Compose**.

SIN LAYOUT

Si en tu proyecto de Android Studio no aparece la carpeta **layout**, hay varias posibilidades. La más común es que estés usando **Jetpack Compose**, donde las interfaces se crean por código y no se utilizan archivos XML de layout.

1. Comprueba si tu proyecto usa Jetpack Compose

Abre tu archivo MainActivity.

Si ves algo parecido a esto:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            MiAplicacionTheme {
                Greeting("Android")
            }
        }
    }
}
```

o funciones con la anotación:

```
@Composable
fun Greeting(nombre: String) {
    Text(text = "Hola $nombre!")
}
```

entonces tu proyecto está usando **Jetpack Compose** y es normal que no exista la carpeta `res/layout`.

2. Comprueba la vista del proyecto

A veces la carpeta sí existe pero está oculta.

En el panel izquierdo de Android Studio:

1. Busca el desplegable superior.
2. Cambia de:

Android

a:

Project

Luego navega a:

```
app
└─ src
    └─ main
        └─ res
            └─ layout
```

3. Si la carpeta no existe, créala manualmente

Haz clic derecho sobre:

```
res
```

Luego:

```
New
→ Android Resource Directory
```

Configura:

```
Resource type: layout
```

Pulsa **OK**.

Android Studio creará:

```
res
└─ layout
```

4. Crear un archivo XML de layout

Después:

```
layout
→ Click derecho
→ New
→ Layout Resource File
```

Por ejemplo:

```
activity_main.xml
```

Ejemplo básico

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hola Mundo"/>
```

```
</LinearLayout>
```

5. Conectar el XML con la Activity

Si trabajas con XML, en `MainActivity` debes tener algo parecido a:

Kotlin

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        setContentView(R.layout.activity_main)  
    }  
}
```

¿Qué significa esta línea?

```
setContentView(R.layout.activity_main)
```

- `R` → clase generada automáticamente por Android.
- `layout` → carpeta de recursos de interfaz.
- `activity_main` → archivo `activity_main.xml`.

Le estás diciendo a Android:

"Muestra en pantalla el contenido definido en `activity_main.xml`".

6. Si tu profesor pide "layouts por programación"

Ni siquiera necesitas la carpeta `layout`.

Puedes crear toda la interfaz desde código:

Kotlin

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
  
    val layout = LinearLayout(this)  
  
    layout.orientation = LinearLayout.VERTICAL  
  
    val texto = TextView(this)  
    texto.text = "Hola Clase"  
  
    layout.addView(texto)  
  
    setContentView(layout)  
}
```

Aquí:

- Se crea el `LinearLayout`.
- Se crea el `TextView`.
- Se añade el texto al layout.
- El layout se convierte en la pantalla mediante `setContentView()`.

No interviene ningún XML ni ninguna carpeta `layout`.

FDO. ROSA MARIA LOPEZ